Academia Java Backend Developer

Ementa Oficial, Visão Geral Aula a Aula e Guia de Orientação para Slides

Stack: Java 21 • Spring Boot 4.1.x • IntelliJ IDEA • Projeto MyMovies • 28 Horas (8 Aulas de 3h30)

1. Visão Geral e Estrutura da Formação

O curso de **Java Backend com Spring Boot** é um programa prático de 28 horas focado na construção, segurança, teste e implantação de APIs RESTful de nível corporativo. A formação utiliza um projeto transversal integrador chamado **MyMovies** (catálogo pessoal de filmes integrado à API pública do TMDB - The Movie Database), que evolui incrementalmente aula a aula.

Pilares Pedagógicos e Níveis de Profundidade na Turma:

- **Core Track:** Conhecimento e prática obrigatórios para todos os alunos. Garante a construção do fluxo end-to-end da API.
- **Deep Dive:** Aprofundamento técnico em comportamento do framework, transações, concorrência e modos de falha.
- **Architect View:** Análise de fronteiras, ownership de contratos, baixo acoplamento e correspondência conceitual com nuvem.
- **AI Tutor (IA Orientada):** Uso da Inteligência Artificial como assistente de diagnóstico e explicação, proibindo a geração massiva sem verificação.

2. Legenda de Tópicos Memos (Orientação de Conteúdo nos Slides)

Para facilitar a localização visual nos slides do curso e situar os alunos durante a exposição teórica e prática, cada bloco de conteúdo é identificado pelos seguintes **marcadores de contexto (Memos)**:

Marcador / Tag Memo	Foco do Conteúdo	Aplicação Pedagógica em Sala
CONCEPT	Ideia ou fundamento teórico central	Apresentação dos princípios arquiteturais e protocolos.
CODE	Trecho focado de código Java / Spring	Análise de sintaxe, anotações e estruturas do projeto.
JAVA REFRESHER	Revisão pontual do Java SE	Lembrete de recursos do Java 21 (Streams, Records, Optionals).
DEEP DIVE	Aprofundamento técnico opcional	Comportamento interno do Spring, transações e internals.

ARCHITECT VIEW	Fronteiras e Trade-offs	Decisões de arquitetura, acoplamento e modos de falha.
AI TUTOR	Prompt para aprendizado guiado	Exercício de diagnóstico com IA sem gerar código pronto.
LAB	Desafio prático do aluno	Enunciado do exercício hands-on e critérios de aceitação.
CHECKPOINT	Validação e Git Tag oficial	Verificação do Definition of Done antes do commit e tag.

3. Mapa Geral da Ementa (Visão Macro 8 Aulas)

Aula	Tema Central	Evolução da API MyMovies	Tag de Checkpoint Git
Aula 1	Spring Boot & REST Básico	Primeira exposição HTTP, GET /api/movies e busca por ID	checkpoint/aula-01-rest
Aula 2	API Design, DTOs & Validação	CRUD completo com desacoplamento DTO e Bean Validation	checkpoint/aula-02-api-design
Aula 3	Persistência com JPA & Hibernate	Migração de memória para Banco H2 com Spring Data JPA	checkpoint/aula-03-jpa
Aula 4	Integração Externa (TMDB)	Consumo da API pública do TMDB via RestClient e importação	checkpoint/aula-04-tmdb
Aula 5	Qualidade, Erros & Testes	Tratamento global de exceções, Logs, Testes e Actuator	checkpoint/aula-05-quality
Aula 6	Spring Security, JWT & OAuth2	Proteção de endpoints como OAuth2 Resource Server	checkpoint/aula-06-security
Aula 7	Empacotamento & Container	Build do Executable JAR e criação de imagem Docker/OCI	checkpoint/aula-07-deployable
Aula 8	Kubernetes Local (Minikube)	Implantação no Minikube com Deployment, Service & Secrets	checkpoint/aula-08-final

4. Detalhamento da Ementa Aula a Aula

Aula 1 — Spring Boot e Fundamentos de REST

Resultado Esperado (Definition of Done): Aplicação rodando e expondo dados em formato JSON através de interface HTTP.
Tag do Checkpoint Git: *checkpoint/aula-01-rest*

Tópicos e Conceitos Abordados:

- Conceitos de HTTP (Request/Response), Verbos e Arquitetura REST voltada a Recursos.
- Diferenciação pedagógica entre Java (Linguagem), Spring (Framework) e Spring Boot (Bootstrapping).
- Criação de Controllers com @RestController, @RequestMapping, @GetMapping.

- Passagem de parâmetros: @PathVariable para identificadores e @RequestParam para filtros.
- Serialização automática de objetos Java para JSON através do Jackson.
- Configuração e execução no IntelliJ IDEA e build com Maven Wrapper (./mvnw).

Ancoragem de Slides (Marcadores Memos):

- CONCEPT: A chamada local vira endpoint de rede ao cruzar a fronteira da JVM.
- CODE: Sintaxe básica do Controller e mapeamento de rotas GET.
- JAVA REFRESHER: Manipulação de coleções com Streams e Optionals.
- LAB: Implementação do endpoint GET /api/movies/{id}.
- CHECKPOINT: Validação do Definition of Done e commit da tag checkpoint/aula-01-rest.

Aula 2 — API Design, DTOs e Validação de Entrada

Resultado Esperado (Definition of Done): CRUD funcional garantindo o isolamento entre o modelo interno e o contrato público da API.
Tag do Checkpoint Git: *checkpoint/aula-02-api-design*

Tópicos e Conceitos Abordados:

- Boas práticas de API Design: Nomenclatura de recursos, pluralização e códigos de status HTTP (200, 201, 204, 400, 404).
- Diferenças semânticas entre operações idempotentes e não-idempotentes (POST vs. PUT vs. PATCH vs. DELETE).
- Padrão DTO (Data Transfer Object): Desacoplamento do contrato externo (MovieRequest e MovieResponse).
- Validação de entrada com Bean Validation (@Valid, @NotNull, @NotBlank, @Size, @Min).
- Introdução ao tratamento e formatação de erros de validação na resposta HTTP.

Ancoragem de Slides (Marcadores Memos):

- CONCEPT: O DTO é a fronteira de proteção do modelo de domínio interno.
- CODE: Anotações do Bean Validation nos Records/Classes de DTO.
- ARCHITECT VIEW: Análise de trade-offs entre PUT (substituição total) e PATCH (atualização parcial).
- LAB: Construção do fluxo POST /api/movies com validação e retorno do HTTP 201 Created.
- CHECKPOINT: Tag oficial checkpoint/aula-02-api-design.

Aula 3 — Persistência de Dados com JPA e Hibernate

Resultado Esperado (Definition of Done): Migração do armazenamento em memória para persistência relacional com Spring Data JPA.
Tag do Checkpoint Git: *checkpoint/aula-03-jpa*

Tópicos e Conceitos Abordados:

- Mapeamento Objeto-Relacional (ORM) com Jakarta Persistence (JPA) e Hibernate.
- Criação de Entidades JPA (@Entity, @Table, @Id, @GeneratedValue, @Column).
- Criação de Repositórios com Spring Data JPA estendendo JpaRepository.
- Derivação automática de consultas (Query Methods) por convenção de nomenclatura.
- Anotação @Transactional e definição de limites de transação na camada de Serviço.
- Configuração do banco de dados em memória H2 para desenvolvimento rápido.

Ancoragem de Slides (Marcadores Memos):

- CONCEPT: Separação clara de responsabilidades: Controller -> Service -> Repository -> Entity.
- CODE: Estrutura da Entidade Movie e do MovieRepository.
- DEEP DIVE: Problema das N+1 consultas e comportamento Lazy vs Eager loading.
- LAB: Migração de todas as operações do CRUD de memória para o banco H2.
- CHECKPOINT: Tag oficial checkpoint/aula-03-jpa.

Aula 4 — Integração com Serviços Externos (API do TMDB)

Resultado Esperado (Definition of Done): Integração funcional com a API pública do TMDB mantendo o controle do contrato próprio da aplicação.
Tag do Checkpoint Git: *checkpoint/aula-04-tmdb*

Tópicos e Conceitos Abordados:

- Uso do Spring RestClient para consumo síncrono e moderno de APIs HTTP externas.
- Gestão de autenticação via API Key / Bearer Tokens em requisições de saída.
- Mapeamento de DTOs externos (TMDB DTO) para entidades internas da MyMovies API (Contract Ownership).
- Tratamento de falhas externas e resiliência: Timeouts, HTTP 404 e erros 5xx da API parceira.
- Funcionalidade de busca de filmes no TMDB e importação selecionada para a base local.

Ancoragem de Slides (Marcadores Memos):

- CONCEPT: Padrão Contract Ownership — nunca vazar o JSON de uma API externa para o seu cliente.
- CODE: Configuração do RestClient e chamada de busca ao TMDB.
- ARCHITECT VIEW: Modos de falha ao depender de serviços de terceiros e estratégias de fallback.
- LAB: Implementação da busca externa e botão de importação para o acervo MyMovies.
- CHECKPOINT: Tag oficial checkpoint/aula-04-tmdb.

Aula 5 — Qualidade, Tratamento de Erros, Testes e Observabilidade

Resultado Esperado (Definition of Done): API resiliente, com respostas de erro padronizadas, testes automatizados e monitoramento de saúde.
Tag do Checkpoint Git: *checkpoint/aula-05-quality*

Tópicos e Conceitos Abordados:

- Tratamento global de exceções com @RestControllerAdvice e padronização (ProblemDetail / ApiError).
- Práticas de Logging corporativo e mascaramento de dados sensíveis (Sanitization).
- Construção de Testes Unitários de serviço e Testes de Integração de API com MockMvc.
- Observabilidade básica e Health Checks utilizando o Spring Boot Actuator (/actuator/health).
- Documentação interativa de API com OpenAPI/Swagger (conceitos básicos).

Ancoragem de Slides (Marcadores Memos):

- CONCEPT: Erros amigáveis para o cliente públicos vs. logs detalhados para o desenvolvedor interno.
- CODE: Estrutura da classe @RestControllerAdvice e métodos @ExceptionHandler.
- AI TUTOR: Diagnóstico de falha de teste com auxílio do assistente de IA.
- LAB: Cobertura de cenários de sucesso e falha (404/400) com testes automatizados.
- CHECKPOINT: Tag oficial checkpoint/aula-05-quality.

Aula 6 — Segurança com Spring Security, JWT e OAuth2

Resultado Esperado (Definition of Done): API protegida como OAuth2 Resource Server exigindo tokens JWT válidos.
Tag do Checkpoint Git: *checkpoint/aula-06-security*

Tópicos e Conceitos Abordados:

- Diferença fundamental entre Autenticação (AuthN) e Autorização (AuthZ).
- Arquitetura de Segurança com Spring Security e a cadeia de filtros (SecurityFilterChain).
- Estrutura e anatomia do JSON Web Token (JWT): Header, Payload, Signature, Claims e Validade.
- Configuração da aplicação como OAuth2 Resource Server para validação de assinatura de tokens.
- Controle de acesso granular baseado em perfis/autoridades (@PreAuthorize, hasAuthority).

Ancoragem de Slides (Marcadores Memos):

- CONCEPT: JWT é um formato de token assinado, não um protocolo de autorização por si só.
- CODE: Configuração da SecurityFilterChain definindo rotas públicas e protegidas.
- ARCHITECT VIEW: Limites de confiança (Trust Boundary), chave pública/JWKS e tempo de expiração.

- LAB: Proteção das rotas de criação/exclusão e liberação da consulta pública de filmes.
- CHECKPOINT: Tag oficial checkpoint/aula-06-security.

Aula 7 — Empacotamento, Configuração Externalizada e Containers (Docker)

Resultado Esperado (Definition of Done): Geração de imagem Docker otimizada executável em qualquer ambiente mantendo segredos fora da imagem.
Tag do Checkpoint Git: *checkpoint/aula-07-deployable*

Tópicos e Conceitos Abordados:

- Ciclo de Build do Maven: Testes, compilação e empacotamento do Executable Fat JAR.
- Execução via linha de comando (java -jar) fora do ambiente da IDE.
- Externalização de configurações utilizando Variáveis de Ambiente e Perfis de Aplicação (Spring Profiles).
- Fundamentos de Containers: Imagem vs. Container, Dockerfile e estágios de build (Multi-stage Build).
- Boas práticas de segurança em imagens: Execução como usuário não-root e injeção de segredos.

Ancoragem de Slides (Marcadores Memos):

- CONCEPT: Imagem Container = Runtime portátil estático que roda igual em Dev, Staging e Prod.
- CODE: Construção do Dockerfile otimizado para o Spring Boot.
- ARCHITECT VIEW: O perigo de depender do sistema de arquivos local ao escalar instâncias.
- LAB: Gerar o JAR, criar a imagem Docker local e rodar a aplicação em container.
- CHECKPOINT: Tag oficial checkpoint/aula-07-deployable.

Aula 8 — Implantação em Kubernetes Local (Minikube)

Resultado Esperado (Definition of Done): Aplicação implantada em um cluster Kubernetes local escalada com 2 réplicas funcionais.
Tag do Checkpoint Git: *checkpoint/aula-08-final*

Tópicos e Conceitos Abordados:

- Visão macro de Orquestração: Nós, Pods, Deployments e Serviços no Kubernetes.
- Instalação e operação do Minikube e utilitário de linha de comando kubectl.
- Gestão de configurações e segredos com ConfigMaps e Secrets do Kubernetes.
- Verificação contínua de saúde da aplicação no cluster (Liveness e Readiness Probes).
- Escalonamento horizontal de réplicas e teste de resiliência e balanceamento local.
- Correspondência conceitual entre o ambiente local (Minikube) e rúnicos gerenciados na nuvem.

Ancoragem de Slides (Marcadores Memos):

- CONCEPT: Orquestração garante que se um container falhar, outro é criado automaticamente.
- CODE: Escrita dos manifestos YAML (deployment.yaml e service.yaml).
- ARCHITECT VIEW: Correspondência de conceitos locais (Minikube) com serviços Cloud (AWS EKS, Azure AKS).
- LAB: Subir a aplicação no Minikube, injetar a chave do TMDB via Secret e escalar para 2 réplicas.
- CHECKPOINT: Tag final do curso checkpoint/aula-08-final.